

Statistical and Machine Learning

Dimension reduction

Katarzyna Reluga

Dimension reduction

- ▶ Introduction
- ▶ PCA
- ▶ Factor analysis
 - ▶ Factor analysis and PCA: PPCA
- ▶ Manifold learning
 - ▶ Multi-dimensional scaling
 - ▶ tSNE
 - ▶ Manifold learning and PCA: kernel PCA

Dimension reduction

- ▶ **Introduction**
- ▶ PCA
- ▶ Factor analysis
 - ▶ Factor analysis and PCA: PPCA
- ▶ Manifold learning
 - ▶ Multi-dimensional scaling
 - ▶ tSNE
 - ▶ Manifold learning and PCA: kernel PCA

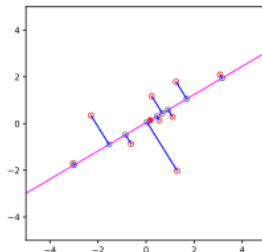
Dimension reduction

Dimensionality reduction: learn a mapping from high-dimensional visible space $\mathbf{x} \in \mathbb{R}^D$ to a low-dimensional latent space $\mathbf{z} \in \mathbb{R}^L$

Two types of mapping:

- ▶ **Parametric model:** $\mathbf{z} = f(\mathbf{x}; \theta)$ – applicable to any input
- ▶ **Non-parametric embedding:** compute $\mathbf{z}_n$ for each $\mathbf{x}_n$ – mostly used for data visualisation

Use cases: a) data visualisation, b) pre-processing step for other learning algorithms (e.g. reduce dimensionality first, then learn a linear classifier)



Dimension reduction

- ▶ Introduction
- ▶ **PCA**
- ▶ Factor analysis
 - ▶ Factor analysis and PCA: PPCA
- ▶ Manifold learning
 - ▶ Multi-dimensional scaling
 - ▶ tSNE
 - ▶ Manifold learning and PCA: kernel PCA

Principal Components Analysis (PCA)

The simplest and most widely used form of dimensionality reduction.

Basic idea: find a linear and orthogonal projection of $\mathbf{x} \in \mathbb{R}^D$ to $\mathbf{z} \in \mathbb{R}^L$ such that the low-dimensional representation is a “good approximation” to the original data.

- ▶ **Encode** $\mathbf{x}$ to get $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$
- ▶ **Decode** $\mathbf{z}$ to get $\hat{\mathbf{x}} = \mathbf{W}\mathbf{z}$
- ▶ We want $\hat{\mathbf{x}}$ to be close to $\mathbf{x}$ in ℓ_2 distance

Reconstruction Error

Define the **reconstruction error** or **distortion**:

$$\mathcal{L}(\mathbf{W}) \triangleq \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \text{decode}(\text{encode}(\mathbf{x}_n; \mathbf{W}); \mathbf{W})\|_2^2$$

where the encode and decode stages are both linear maps.

Optimal solution: $\hat{\mathbf{W}} = \mathbf{U}_L$, where $\mathbf{U}_L$ contains the L eigenvectors with largest eigenvalues of the empirical covariance matrix

$$\hat{\mathbf{\Sigma}} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top = \frac{1}{N} \mathbf{X}_c^\top \mathbf{X}_c$$

where $\mathbf{X}_c$ is a centered version of the $N \times D$ design matrix.

Derivation: Setup

Dataset $\mathcal{D} = \{\mathbf{x}_n : n = 1 : N\}$, $\mathbf{x}_n \in \mathbb{R}^D$. Assume $\bar{\mathbf{x}} = \mathbf{0}$ (center the data).

Approximate each $\mathbf{x}_n$ by a low-dimensional representation $\mathbf{z}_n \in \mathbb{R}^L$:

$$\mathbf{x}_n \approx \sum_{k=1}^L z_{nk} \mathbf{w}_k$$

where $\mathbf{w}_k \in \mathbb{R}^D$ are basis functions and $\mathbf{z}_n \in \mathbb{R}^L$ are **latent vectors** (their components are the **latent factors**).

Reconstruction error:

$$\mathcal{L}(\mathbf{W}, \mathbf{Z}) = \frac{1}{N} \|\mathbf{X} - \mathbf{Z}\mathbf{W}^\top\|_F^2 = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - \mathbf{W}\mathbf{z}_n\|^2$$

Minimise subject to $\mathbf{W}$ being an **orthogonal** matrix.

Derivation: Base Case (1D) – finding the optimal z_{n1}^*

Task: find the best $\mathbf{w}_1 \in \mathbb{R}^D$.

Let $\tilde{\mathbf{z}}_1 = [z_{11}, \dots, z_{N1}]^\top \in \mathbb{R}^N$. The reconstruction error is:

$$\begin{aligned}\mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1) &= \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{n1} \mathbf{w}_1\|^2 = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - z_{n1} \mathbf{w}_1)^\top (\mathbf{x}_n - z_{n1} \mathbf{w}_1) \\ &= \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - 2z_{n1} \mathbf{w}_1^\top \mathbf{x}_n + z_{n1}^2 \mathbf{w}_1^\top \mathbf{w}_1] \\ &= \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - 2z_{n1} \mathbf{w}_1^\top \mathbf{x}_n + z_{n1}^2] (\mathbf{w}_1^\top \mathbf{w}_1 = 1, \text{ orthonorm.})\end{aligned}$$

Taking the derivative w.r.t. z_{n1} and setting to zero:

$$\frac{\partial}{\partial z_{n1}} \mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1) = \frac{1}{N} [-2\mathbf{w}_1^\top \mathbf{x}_n + 2z_{n1}] = 0 \quad \Rightarrow \quad \boxed{z_{n1}^* = \mathbf{w}_1^\top \mathbf{x}_n}$$

The optimal embedding is obtained by **orthogonally projecting** the data onto $\mathbf{w}_1$.

Derivation: Base Case (1D) – finding the optimal $\mathbf{w}_1$

Plugging $z_{n1}^* = \mathbf{w}_1^\top \mathbf{x}_n$ back into the loss:

$$\mathcal{L}(\mathbf{w}_1) = \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - z_{n1}^2] = \text{const} - \frac{1}{N} \sum_{n=1}^N z_{n1}^2 = -\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1$$

since $\mathcal{L}(\mathbf{w}_1) = -\frac{1}{N} \sum_n \mathbf{w}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_1 = -\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1$, where $\hat{\Sigma}$ is the empirical covariance matrix.

To prevent $\|\mathbf{w}_1\| \rightarrow \infty$, impose $\|\mathbf{w}_1\| = 1$ and optimize the Lagrangian

$$\tilde{\mathcal{L}}(\mathbf{w}_1) = \mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 - \lambda_1 (\mathbf{w}_1^\top \mathbf{w}_1 - 1)$$

where λ_1 is a Lagrange multiplier.

Derivation: Base Case (1D) – finding the optimal $\mathbf{w}_1$

Taking the derivative of $\tilde{\mathcal{L}}(\mathbf{w}_1)$ and setting to zero:

$$\frac{\partial}{\partial \mathbf{w}_1} \tilde{\mathcal{L}}(\mathbf{w}_1) = 2\hat{\Sigma}\mathbf{w}_1 - 2\lambda_1\mathbf{w}_1 = 0 \quad \Rightarrow \quad \hat{\Sigma}\mathbf{w}_1 = \lambda_1\mathbf{w}_1$$

So $\mathbf{w}_1$ is an **eigenvector** of $\hat{\Sigma}$. Left-multiplying by $\mathbf{w}_1^\top$ (using $\mathbf{w}_1^\top \mathbf{w}_1 = 1$):

$$\mathbf{w}_1^\top \hat{\Sigma} \mathbf{w}_1 = \lambda_1$$

To **minimise** $\mathcal{L}$ we **maximise** $\lambda_1 \Rightarrow$ pick the eigenvector with the **largest eigenvalue**.

Derivation: variance interpretation

Since the data is centered, $\mathbb{E}[z_{n1}] = \mathbb{E}[\mathbf{x}_n]^\top \mathbf{w}_1 = 0$.

The variance of the projected data is:

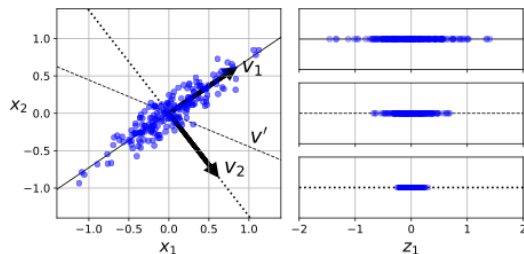
$$\mathbb{V}[\tilde{z}_1] = \mathbb{E}[\tilde{z}_1^2] - (\mathbb{E}[\tilde{z}_1])^2 = \frac{1}{N} \sum_{n=1}^N z_{n1}^2 = -\mathcal{L}(\mathbf{w}_1) + \text{const}$$

Therefore:

$$\arg \min_{\mathbf{w}_1} \mathcal{L}(\mathbf{w}_1) = \arg \max_{\mathbf{w}_1} \mathbb{V}[\tilde{z}_1(\mathbf{w}_1)]$$

Key insight: minimising reconstruction error $\equiv$ maximising the variance of the projected data. PCA finds the **directions of maximal variance**.

Derivation: illustration of variance interpretation



- ▶ v_1 is the first principal component, which maximizes the variance of the projection.
- ▶ v_2 is the second principal component which is direction orthogonal to v_1
- ▶ v' is some other vector in between v_1 and v_2 .

Derivation: induction step – finding the optimal z_{n2}^*

Find $\mathbf{w}_2$ to further minimise error, subject to $\mathbf{w}_1^\top \mathbf{w}_2 = 0$ and $\mathbf{w}_2^\top \mathbf{w}_2 = 1$. The error is:

$$\mathcal{L}(\mathbf{w}_1, \tilde{\mathbf{z}}_1, \mathbf{w}_2, \tilde{\mathbf{z}}_2) = \frac{1}{N} \sum_{n=1}^N \|\mathbf{x}_n - z_{n1} \mathbf{w}_1 - z_{n2} \mathbf{w}_2\|^2$$

Optimising w.r.t. $\mathbf{w}_1$ and z_{n1} gives the **same solution as before**:
 $z_{n1}^* = \mathbf{w}_1^\top \mathbf{x}_n$.

Taking the derivative w.r.t. z_{n2} and setting to zero:

$$\frac{\partial \mathcal{L}}{\partial z_{n2}} = 0 \quad \Rightarrow \quad z_{n2}^* = \mathbf{w}_2^\top \mathbf{x}_n$$

Substituting both z_{n1}^* and z_{n2}^* back in yields:

$$\mathcal{L}(\mathbf{w}_2) = \frac{1}{N} \sum_{n=1}^N [\mathbf{x}_n^\top \mathbf{x}_n - \mathbf{w}_1^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_1 - \mathbf{w}_2^\top \mathbf{x}_n \mathbf{x}_n^\top \mathbf{w}_2] = \text{const} - \mathbf{w}_2^\top \hat{\mathbf{\Sigma}} \mathbf{w}_2$$

Derivation: induction step – finding the optimal $\mathbf{w}_2$

Dropping the constant term and adding the constraints via Lagrange multipliers λ_2 and λ_{12} :

$$\tilde{\mathcal{L}}(\mathbf{w}_2) = -\mathbf{w}_2^\top \hat{\Sigma} \mathbf{w}_2 + \lambda_2(\mathbf{w}_2^\top \mathbf{w}_2 - 1) + \lambda_{12}(\mathbf{w}_2^\top \mathbf{w}_1 - 0)$$

Taking the derivative and setting to zero:

$$\frac{\partial \tilde{\mathcal{L}}}{\partial \mathbf{w}_2} = -2\hat{\Sigma} \mathbf{w}_2 + 2\lambda_2 \mathbf{w}_2 + \lambda_{12} \mathbf{w}_1 = \mathbf{0}$$

Left-multiplying by $\mathbf{w}_1^\top$ and using $\mathbf{w}_1^\top \mathbf{w}_2 = 0$ and $\hat{\Sigma} \mathbf{w}_2 = \lambda_1 \mathbf{w}_2$ gives $\lambda_{12} = 0$, so:

$$\hat{\Sigma} \mathbf{w}_2 = \lambda_2 \mathbf{w}_2$$

$\Rightarrow \mathbf{w}_2$ is an eigenvector of $\hat{\Sigma}$, and since $\mathbf{w}_2 \perp \mathbf{w}_1$, we pick the eigenvector with the **second largest eigenvalue** λ_2 .

The proof continues by induction $\Rightarrow \hat{\mathbf{W}} = \mathbf{U}_L$.

Computational issues: covariance vs. correlation

We have been working with the eigendecomposition of the **covariance** matrix.

Problem: PCA can be “misled” by directions with high variance purely because of the measurement scale.

Solution: use the **correlation** matrix instead, i.e. standardise the data first:

$$\mathbf{x}_n^* = \frac{\mathbf{x}_n - \mu}{\sigma}$$

- ▶ Equivalent to applying PCA to the correlation matrix
- ▶ Results look much more natural and interpretable
- ▶ Especially important when variables are on very different scales

Computational Issues: high-dimensional data

PCA $\rightarrow$ find the eigenvectors of the $D \times D$ covariance matrix $\mathbf{X}^\top \mathbf{X}$.

Problem: if $D > N$, this is expensive. Instead, work with the $N \times N$ **Gram matrix** $\mathbf{X}\mathbf{X}^\top$.

Let $\mathbf{U}$ be the orthonormal eigenvectors of $\mathbf{X}\mathbf{X}^\top$ with eigenvalues $\mathbf{\Lambda}$, so $(\mathbf{X}\mathbf{X}^\top)\mathbf{U} = \mathbf{U}\mathbf{\Lambda}$. Pre-multiplying by $\mathbf{X}^\top$:

$$(\mathbf{X}^\top \mathbf{X})(\mathbf{X}^\top \mathbf{U}) = (\mathbf{X}^\top \mathbf{U})\mathbf{\Lambda}$$

So the eigenvectors of $\mathbf{X}^\top \mathbf{X}$ are $\mathbf{V} = \mathbf{X}^\top \mathbf{U}$, with the same eigenvalues $\mathbf{\Lambda}$.

$\mathbf{V}$ not normalised $\rightarrow \|\mathbf{v}_j\|^2 = \mathbf{u}_j^\top \mathbf{X}\mathbf{X}^\top \mathbf{u}_j = \lambda_j \mathbf{u}_j^\top \mathbf{u}_j = \lambda_j$

Hence, the normalised eigenvectors are:

$$\mathbf{V} = \mathbf{X}^\top \mathbf{U} \mathbf{\Lambda}^{-\frac{1}{2}}$$

Reminder: Singular Value Decomposition (SVD)

Any real $N \times D$ matrix $\mathbf{X}$ can be decomposed as:

$$\mathbf{X} = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \cdots + \sigma_r \mathbf{u}_r \mathbf{v}_r^\top$$

where $r = \min(N, D)$ and:

- ▶ $\mathbf{U}_X$ is $N \times N$ with orthonormal columns: left singular vectors ($\mathbf{U}_X^\top \mathbf{U}_X = \mathbf{I}_N$)
- ▶ $\mathbf{V}_X$ is $D \times D$ with orthonormal rows and columns: right singular vectors ($\mathbf{V}_X^\top \mathbf{V}_X = \mathbf{V}_X \mathbf{V}_X^\top = \mathbf{I}_D$)
- ▶ $\mathbf{S}_X$ is $N \times D$ with **singular values** $\sigma_i \geq 0$ on the diagonal

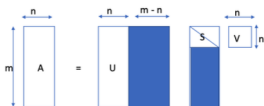
Reminder: Singular Value Decomposition (SVD)

If $m > n$ (see figure below), there are at most n singular values, so the last $m - n$ columns of U are irrelevant (since they will be multiplied by 0)

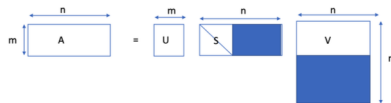
The **economy (thin) SVD** discards the zero-padded parts: if $N > D$, only compute the first D columns of $\mathbf{U}_X$.

The **L -truncated SVD** $\mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top \approx \mathbf{X}$ keeps only the top L singular values/vectors – the best rank- L approximation to $\mathbf{X}$.

Cost of full SVD: $O(\min(ND^2, N^2D))$.



(a)



(b)

Computing PCA via SVD

Let $\mathbf{U}_\Sigma \mathbf{\Lambda}_\Sigma \mathbf{U}_\Sigma^\top$ be the top- L eigendecomposition of $\hat{\Sigma} \propto \mathbf{X}^\top \mathbf{X}$.

Recall that $\mathbf{W} = \mathbf{U}_\Sigma$.

Let $\mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top \approx \mathbf{X}$ be the L -truncated SVD of $\mathbf{X}$. The right singular vectors of $\mathbf{X}$ are the eigenvectors of $\mathbf{X}^\top \mathbf{X}$, so $\mathbf{V}_X = \mathbf{U}_\Sigma = \mathbf{W}$, and $\lambda_k = s_k^2/N$.

The **PC scores** (projected points) are:

$$\mathbf{Z} = \mathbf{XW} = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top \mathbf{V}_X = \mathbf{U}_X \mathbf{S}_X$$

The **approximate reconstruction** is:

$$\hat{\mathbf{X}} = \mathbf{ZW}^\top = \mathbf{U}_X \mathbf{S}_X \mathbf{V}_X^\top$$

This is precisely a truncated SVD approximation. PCA via SVD is often preferable computationally.

Choosing L : reconstruction error

The reconstruction error on $\mathcal{D}$ when using L dimensions is:

$$\mathcal{L}_L = \frac{1}{|\mathcal{D}|} \sum_{n \in \mathcal{D}} \|\mathbf{x}_n - \hat{\mathbf{x}}_n\|^2$$

where $\hat{\mathbf{x}}_n = \mathbf{W}\mathbf{z}_n + \boldsymbol{\mu}$, $\mathbf{z}_n = \mathbf{W}^\top(\mathbf{x}_n - \boldsymbol{\mu})$. One can show (left without the proof):

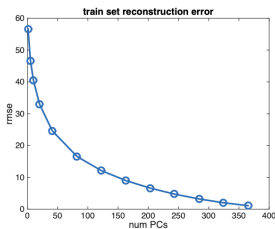
$$\mathcal{L}_L = \sum_{j=L+1}^D \lambda_j$$

So $\mathcal{L}_L$ equals the sum of the **discarded** eigenvalues - as L increases, the discarded eigenvalues get smaller $\Rightarrow$ reconstruction error decreases monotonically.

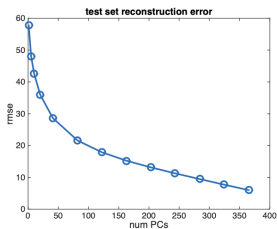
Choosing L : illustration of reconstruction error

On the training set: $\mathcal{L}_L$ drops off quickly; setting $L = \text{rank}(\mathbf{X})$ gives zero error.

On the test set: the error still keeps going down as L increases – no U-shape! This is because PCA is **not a proper generative model**: more latent dimensions always improve approximation of unseen data. We need other criteria to select L .



(a)



(b)

Reconstruction error on MNIST vs number of latent dimensions used by PCA. (a) Training set. (b) Test set.

Choosing L : scree and fraction of variation plots

A **scree plot** is a plot of the eigenvalues λ_j vs. j in order of decreasing magnitude.

From the eigenvalue decomposition of the error: $\mathcal{L}_L = \sum_{j=L+1}^D \lambda_j$.

As the number of dimensions L increases, the discarded eigenvalues get smaller $\Rightarrow$ reconstruction error decreases monotonically.

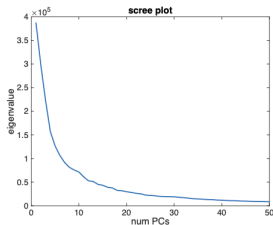
A related quantity is the **fraction of variance explained**:

$$F_L = \frac{\sum_{j=1}^L \lambda_j}{\sum_{j'=1}^{L_{\max}} \lambda_{j'}}$$

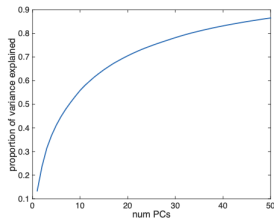
This is the complement of $\mathcal{L}_L$, rescaled to $[0, 1]$ – it increases with L .

In practice: look for an “elbow” in the scree plot where eigenvalues level off, or choose L such that F_L exceeds a threshold (e.g. 90% of variance explained).

Choosing L : scree and fraction of variation plots



(a)



(b)

- (a) Scree plot for training set, corresponding to the last figure.
- (b) Fraction of variance explained.

Choosing L : Profile Likelihood – motivation

Although there is no U-shape, the reconstruction error curve sometimes has a *knee* or *elbow*: the error decreases rapidly for $L < L^*$ and then much more slowly for $L > L^*$.

Intuition: for $L < L^*$ (the true latent dimensionality), each extra component captures real structure $\Rightarrow$ large gain. For $L > L^*$, the model is already complex enough $\Rightarrow$ small gain.

Goal: automate the detection of this change-point in the gradient of the curve.

Method (Zhu & Ghodsi, 2006): treat the sorted eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_{L_{\max}}$ as data and fit a **change-point model** at position L . This applies to both PCA (using eigenvalues) and K-means clustering (using reconstruction errors).

Choosing L : Profile Likelihood – method

For $L = 1 : L^{\max}$, fit a two-group Gaussian model with a **shared variance** (to avoid overfitting when one group is small):

$$\lambda_k \sim \mathcal{N}(\mu_1, \sigma^2) \text{ if } k \leq L, \quad \lambda_k \sim \mathcal{N}(\mu_2, \sigma^2) \text{ if } k > L$$

The MLEs are:

$$\begin{aligned} \mu_1(L) &= \frac{\sum_{k \leq L} \lambda_k}{L}, & \mu_2(L) &= \frac{\sum_{k > L} \lambda_k}{L^{\max} - L} \\ \sigma^2(L) &= \frac{\sum_{k \leq L} (\lambda_k - \mu_1(L))^2 + \sum_{k > L} (\lambda_k - \mu_2(L))^2}{L^{\max}} \end{aligned}$$

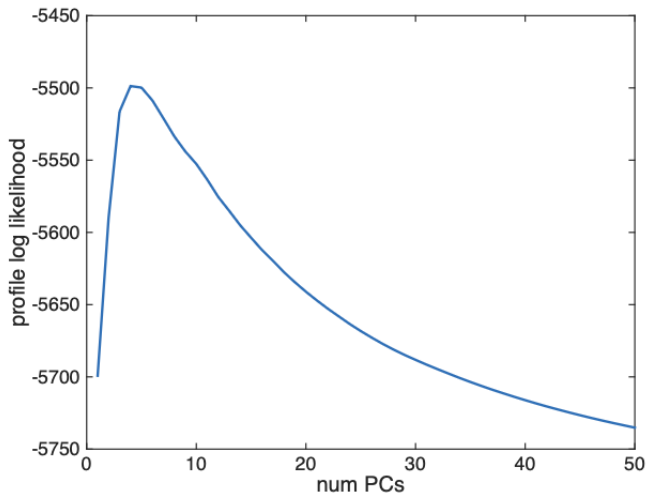
Evaluate the **profile log-likelihood** for each L :

$$\ell(L) = \sum_{k=1}^L \log \mathcal{N}(\lambda_k \mid \mu_1(L), \sigma^2(L)) + \sum_{k=L+1}^{L^{\max}} \log \mathcal{N}(\lambda_k \mid \mu_2(L), \sigma^2(L))$$

Choose $L^* = \arg \max_L \ell(L)$ – the peak is well determined.

Choosing L : Profile Likelihood – illustration

Profile likelihood corresponding to PCA.



Dimension reduction

- ▶ Introduction
- ▶ PCA
- ▶ **Factor analysis**
 - ▶ Factor analysis and PCA: PPCA
- ▶ Manifold learning
 - ▶ Multi-dimensional scaling
 - ▶ tSNE
 - ▶ Manifold learning and PCA: kernel PCA

Factor Analysis

PCA computes a linear low-dimensional representation of data, but it is **not based on a probabilistic model**.

Factor analysis (FA) is a generalisation of PCA based on a proper generative model.

Advantages of having a probabilistic model:

- ▶ Can be used as a building block for more complex models (e.g. mixtures of FA, nonlinear FA)
- ▶ PCA can be recovered as a special limiting case of FA

Generative Model

Factor analysis is a linear-Gaussian latent variable model:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$$

$$p(\mathbf{x} \mid \mathbf{z}, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{x} \mid \mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi})$$

where:

- ▶ $\mathbf{W}$ is a $D \times L$ matrix, the **factor loading matrix**
- ▶ $\boldsymbol{\Psi}$ is a diagonal $D \times D$ covariance matrix

Interpretation: each observation $\mathbf{x}$ is generated by first drawing a latent factor $\mathbf{z}$, linearly mapping it to the visible space via $\mathbf{W}$, and adding Gaussian noise $\boldsymbol{\Psi}$.

The Induced Marginal Distribution

FA can be thought of as a **low-rank version of a Gaussian distribution**. The induced marginal $p(\mathbf{x} \mid \boldsymbol{\theta})$ is itself Gaussian:

$$\begin{aligned} p(\mathbf{x} \mid \boldsymbol{\theta}) &= \int \mathcal{N}(\mathbf{x} \mid \mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi}) \mathcal{N}(\mathbf{z} \mid \boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0) d\mathbf{z} \\ &= \mathcal{N}(\mathbf{x} \mid \mathbf{W}\boldsymbol{\mu}_0 + \boldsymbol{\mu}, \boldsymbol{\Psi} + \mathbf{W}\boldsymbol{\Sigma}_0\mathbf{W}^\top) \end{aligned}$$

Hence $\mathbb{E}[\mathbf{x}] = \mathbf{W}\boldsymbol{\mu}_0 + \boldsymbol{\mu}$ and
 $\text{Cov}[\mathbf{x}] = \mathbf{W}\text{Cov}[\mathbf{z}]\mathbf{W}^\top + \boldsymbol{\Psi} = \mathbf{W}\boldsymbol{\Sigma}_0\mathbf{W}^\top + \boldsymbol{\Psi}$.

Simplifications without loss of generality:

- ▶ Set $\boldsymbol{\mu}_0 = \mathbf{0}$ – absorb $\mathbf{W}\boldsymbol{\mu}_0$ into $\boldsymbol{\mu}$
- ▶ Set $\boldsymbol{\Sigma}_0 = \mathbf{I}$ – absorb a correlated prior using $\tilde{\mathbf{W}} = \mathbf{W}\boldsymbol{\Sigma}_0^{-\frac{1}{2}}$

After these simplifications: $p(\mathbf{z}) = \mathcal{N}(\mathbf{z} \mid \mathbf{0}, \mathbf{I})$,
 $p(\mathbf{x} \mid \mathbf{z}) = \mathcal{N}(\mathbf{x} \mid \mathbf{W}\mathbf{z} + \boldsymbol{\mu}, \boldsymbol{\Psi})$, $p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \mathbf{W}\mathbf{W}^\top + \boldsymbol{\Psi})$

Geometric Intuition: The Spray Can Example

- ▶ Suppose $L = 1$, $D = 2$ and $\Psi = \sigma^2 \mathbf{I}$.
- ▶ Take an isotropic Gaussian “spray can”, representing the likelihood $p(\mathbf{x} \mid \mathbf{z})$
- ▶ “Slide it along” the 1D line defined by $\mathbf{w}\mathbf{z} + \boldsymbol{\mu}$ as the latent prior $\mathbf{z}$ varies

This induces an **elongated (and correlated) Gaussian** in 2D:

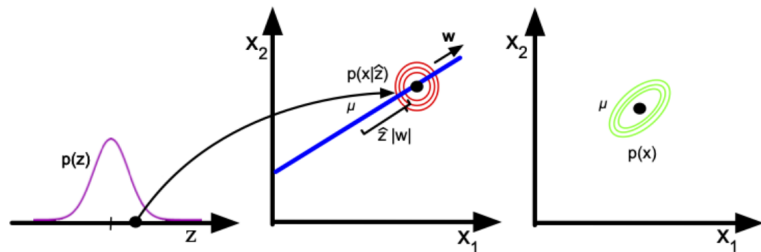
$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \mathbf{w}\mathbf{w}^\top + \sigma^2 \mathbf{I})$$

General principle: FA approximates the covariance matrix of the visible vector using a **low-rank decomposition**:

$$\mathbf{C} = \text{Cov}[\mathbf{x}] = \mathbf{W}\mathbf{W}^\top + \Psi$$

This uses only $O(LD)$ parameters – a flexible compromise between a full covariance Gaussian ($O(D^2)$ parameters) and a diagonal covariance ($O(D)$ parameters).

Illustration: The Spray Can Example



- ▶ A latent factor $z \in \mathbb{R}$ is sampled from $p(z)$.
- ▶ The latent factor is mapped to a 2D offset $\delta = z\mathbf{w}$, with $\mathbf{w} \in \mathbb{R}^2$.
- ▶ This offset shifts the mean to define $p(\mathbf{x} | z) = \mathcal{N}(\mathbf{x} | \boldsymbol{\mu} + \delta, \sigma^2 \mathbf{I})$.
- ▶ Integrating over z moves the circular Gaussian along the axis $\mathbf{w}$, producing elliptical Gaussian contours in $\mathbf{x}$ -space centered at $\boldsymbol{\mu}$.

Why Ψ Must Be Diagonal

From $\mathbf{C} = \mathbf{W}\mathbf{W}^\top + \Psi$, we see that Ψ must be restricted to be diagonal.

Why? Otherwise we could set $\mathbf{W} = \mathbf{0}$ and let Ψ model any covariance directly – thus ignoring the latent factors entirely.

Marginal variance of each visible variable:

$$\mathbb{V}[x_d] = \sum_{k=1}^L w_{dk}^2 + \psi_d$$

- ▶ The first term: variance due to the **common factors**
- ▶ ψ_d : the **uniqueness** – variance specific to dimension d

Posterior Latent Embeddings

Once the FA model is fit (using EM, see next slide), we can compute **probabilistic latent embeddings** using $p(\mathbf{z} \mid \mathbf{x})$.

Using Bayes' rule for Gaussians:

$$p(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z} \mid \mathbf{W}^\top \mathbf{C}^{-1}(\mathbf{x} - \boldsymbol{\mu}), \mathbf{I} - \mathbf{W}^\top \mathbf{C}^{-1} \mathbf{W})$$

where $\mathbf{C} = \mathbf{W}\mathbf{W}^\top + \boldsymbol{\Psi}$.

Key contrast with PCA: unlike PCA's deterministic projection $\mathbf{z} = \mathbf{W}^\top \mathbf{x}$, FA gives a full **posterior distribution** over the latent factors – capturing uncertainty in the embedding.

EM for FA: E Step

We estimate the parameters of an FA model using the **EM algorithm**.

E step: compute the posterior embeddings

$$p(\mathbf{z}_i \mid \mathbf{x}_i, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{z}_i \mid \mathbf{m}_i, \boldsymbol{\Sigma}_i)$$

$$\boldsymbol{\Sigma}_i \triangleq (\mathbf{I}_L + \mathbf{W}^\top \boldsymbol{\Psi}^{-1} \mathbf{W})^{-1}$$

$$\mathbf{m}_i \triangleq \boldsymbol{\Sigma}_i (\mathbf{W}^\top \boldsymbol{\Psi}^{-1} (\mathbf{x}_i - \boldsymbol{\mu}))$$

Plain language: for each data point $\mathbf{x}_i$, compute the mean $\mathbf{m}_i$ and covariance $\boldsymbol{\Sigma}_i$ of its latent embedding, given the current parameter estimates.

EM for FA: M Step – Setup

To estimate $\boldsymbol{\mu}$ and $\mathbf{W}$ together, define:

$$\tilde{\mathbf{W}} = (\mathbf{W}, \boldsymbol{\mu}), \quad \tilde{\mathbf{z}} = (\mathbf{z}, 1)$$

Also define, for each data point i :

$$\mathbf{b}_i \triangleq \mathbb{E}[\tilde{\mathbf{z}} \mid \mathbf{x}_i] = [\mathbf{m}_i; 1]$$

$$\mathbf{C}_i \triangleq \mathbb{E}[\tilde{\mathbf{z}}\tilde{\mathbf{z}}^T \mid \mathbf{x}_i] = \begin{pmatrix} \mathbb{E}[\mathbf{z}\mathbf{z}^T \mid \mathbf{x}_i] & \mathbb{E}[\mathbf{z} \mid \mathbf{x}_i] \\ \mathbb{E}[\mathbf{z} \mid \mathbf{x}_i]^T & 1 \end{pmatrix}$$

These quantities are computed from the E-step results $\mathbf{m}_i$ and $\boldsymbol{\Sigma}_i$, and feed directly into the parameter updates.

EM for FA: M Step – Updates

The M step updates are:

$$\hat{\mathbf{W}} = \left[\sum_i \mathbf{x}_i \mathbf{b}_i^\top \right] \left[\sum_i \mathbf{c}_i \right]^{-1}$$

$$\hat{\Psi} = \frac{1}{N} \text{diag} \left\{ \sum_i (\mathbf{x}_i - \hat{\mathbf{W}} \mathbf{b}_i) \mathbf{x}_i^\top \right\}$$

Plain language: $\hat{\mathbf{W}}$ (containing both $\mathbf{W}$ and μ) is updated as a weighted regression of the data $\mathbf{x}_i$ onto the expected augmented latents $\mathbf{b}_i$. $\hat{\Psi}$ is then updated as the diagonal of the residual covariance.

Note: these are the updates for “vanilla” EM. A much faster ECM-based version exists in the literature.

Unidentifiability of the Parameters

The parameters of an FA model are **unidentifiable**.

Consider a model with weights $\mathbf{W}$ and observation covariance Ψ :

$$\text{Cov}[\mathbf{x}] = \mathbf{W} \mathbb{E}[\mathbf{z}\mathbf{z}^\top] \mathbf{W}^\top + \mathbb{E}[\epsilon\epsilon^\top] = \mathbf{W}\mathbf{W}^\top + \Psi$$

Now consider a **different** model with weights $\tilde{\mathbf{W}} = \mathbf{W}\mathbf{R}$, where $\mathbf{R}$ is an arbitrary orthogonal rotation matrix satisfying $\mathbf{R}\mathbf{R}^\top = \mathbf{I}$.

This rotated model has **the same likelihood**, since:

$$\text{Cov}[\mathbf{x}] = \tilde{\mathbf{W}} \mathbb{E}[\mathbf{z}\mathbf{z}^\top] \tilde{\mathbf{W}}^\top + \mathbb{E}[\epsilon\epsilon^\top] = \mathbf{W}\mathbf{R}\mathbf{R}^\top \mathbf{W}^\top + \Psi = \mathbf{W}\mathbf{W}^\top + \Psi$$

Why Rotation Doesn't Matter

Geometric interpretation: multiplying $\mathbf{W}$ by an orthogonal matrix $\mathbf{R}$ is equivalent to **rotating** $\mathbf{z}$ before generating $\mathbf{x}$.

But $\mathbf{z}$ is drawn from an **isotropic Gaussian** $\mathcal{N}(\mathbf{0}, \mathbf{I})$ – which looks the same from every direction.

Consequence: rotating $\mathbf{z}$ makes no difference to the likelihood.

We cannot uniquely identify $\mathbf{W}$, and therefore cannot uniquely identify the latent factors either.

To break this symmetry, several solutions can be used – we discuss four of them.

Solution 1: Orthonormal Columns

Forcing W to have orthonormal columns.

Perhaps the simplest solution: force **W** to have **orthonormal columns**.

This is the approach adopted by **PCA**.

- ▶ The resulting posterior estimate is then unique, **up to permutation** of the latent dimensions
- ▶ In PCA, this ordering ambiguity is resolved by **sorting the dimensions in order of decreasing eigenvalues**

Solution 2: Lower Triangular $\mathbf{W}$

Forcing $\mathbf{W}$ to be lower triangular.

A popular approach in the Bayesian community: ensure that

- ▶ the first visible feature is generated only by the first latent factor
- ▶ the second visible feature is generated only by the first two latent factors
- ▶ and so on

For example, if $L = 3$ and $D = 4$:

$$\mathbf{W} = \begin{pmatrix} w_{11} & 0 & 0 \\ w_{21} & w_{22} & 0 \\ w_{31} & w_{32} & w_{33} \\ w_{41} & w_{42} & w_{43} \end{pmatrix}$$

We also require $w_{kk} > 0$ for $k = 1:L$.

Solution 2: Lower Triangular **W** – Discussion

The total number of parameters in this constrained matrix is:

$$D + DL - \frac{L(L - 1)}{2}$$

which equals the number of uniquely identifiable parameters in FA.

Disadvantage: the first L visible variables – the **founder variables** – directly affect the interpretation of the latent factors, and so must be chosen carefully.

Solutions 3 & 4: Priors and Rotations

Solution 3: Sparsity-promoting priors on the weights.

Instead of pre-specifying which entries of $\mathbf{W}$ are zero, encourage entries to be (close to) zero using:

- ▶ ℓ_1 regularisation
- ▶ Automatic Relevance Determination (ARD)
- ▶ Spike-and-slab priors

This is called **sparse factor analysis**. It does not necessarily ensure a unique MAP estimate, but encourages interpretable solutions.

Solution 4: Choosing an informative rotation matrix.

Heuristic methods search for a rotation matrix $\mathbf{R}$ that modifies $\mathbf{W}$ to increase interpretability, typically by encouraging (approximate) sparsity. One popular method: **varimax**.

Dimension reduction

- ▶ Introduction
- ▶ PCA
- ▶ Factor analysis
 - ▶ **Factor analysis and PCA: PPCA**
- ▶ Manifold learning
 - ▶ Multi-dimensional scaling
 - ▶ tSNE
 - ▶ Manifold learning and PCA: kernel PCA

Probabilistic PCA

A special case of factor analysis where:

- ▶ $\mathbf{W}$ has **orthonormal columns**
- ▶ $\Psi = \sigma^2 \mathbf{I}$

This model is called **probabilistic principal components analysis (PPCA)**, or **sensible PCA**.

The marginal distribution on the visible variables is

$$p(\mathbf{x}|\boldsymbol{\theta}) = \int \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z}, \sigma^2 \mathbf{I}) \mathcal{N}(\mathbf{z}|\mathbf{0}, \mathbf{I}) d\mathbf{z} = \mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \mathbf{C})$$

where

$$\mathbf{C} = \mathbf{W}\mathbf{W}^\top + \sigma^2 \mathbf{I}$$

Log-likelihood for PPCA

$$\begin{aligned} \log p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2) \\ = -\frac{ND}{2} \log(2\pi) - \frac{N}{2} \log |\mathbf{C}| - \frac{1}{2} \sum_{n=1}^N (\mathbf{x}_n - \boldsymbol{\mu})^\top \mathbf{C}^{-1} (\mathbf{x}_n - \boldsymbol{\mu}) \end{aligned}$$

The MLE for $\boldsymbol{\mu}$ is $\bar{\mathbf{x}}$. Plugging in gives:

$$\log p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2) = -\frac{N}{2} \left[D \log(2\pi) + \log |\mathbf{C}| + \text{tr}(\mathbf{C}^{-1} \mathbf{S}) \right]$$

where $\mathbf{S} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}_n - \bar{\mathbf{x}})(\mathbf{x}_n - \bar{\mathbf{x}})^\top$ is the empirical covariance matrix.

MLE for the loading matrix $\mathbf{W}$

The maximum of this objective must satisfy:

$$\mathbf{W} = \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})^{\frac{1}{2}}\mathbf{R}$$

where:

- ▶ $\mathbf{U}_L$: $D \times L$ matrix of the L eigenvectors of $\mathbf{S}$ with largest eigenvalues
- ▶ $\mathbf{L}_L$: $L \times L$ diagonal matrix of eigenvalues
- ▶ $\mathbf{R}$: arbitrary $L \times L$ orthogonal matrix, WLOG $\mathbf{R} = \mathbf{I}$

In the **noise-free limit** ($\sigma^2 = 0$):

$$\mathbf{W}_{\text{mle}} = \mathbf{U}_L \mathbf{L}_L^{\frac{1}{2}}$$

which is proportional to the PCA solution.

MLE for the observation variance

$$\sigma^2 = \frac{1}{D-L} \sum_{i=L+1}^D \lambda_i$$

- ▶ This is the **average distortion** associated with the discarded dimensions.
- ▶ If $L = D$, the estimated noise is 0, since the model collapses to $\mathbf{z} = \mathbf{x}$.

Efficient computation of the likelihood

To compute $p(\mathbf{X}|\boldsymbol{\mu}, \mathbf{W}, \sigma^2)$, we need $\mathbf{C}^{-1}$ and $\log |\mathbf{C}|$, where $\mathbf{C}$ is $D \times D$.

Using the **matrix inversion lemma**:

$$\mathbf{C}^{-1} = \sigma^{-2} \left[\mathbf{I} - \mathbf{W}\mathbf{M}^{-1}\mathbf{W}^\top \right]$$

where the $L \times L$ matrix $\mathbf{M}$ is given by:

$$\mathbf{M} = \mathbf{W}^\top \mathbf{W} + \sigma^2 \mathbf{I}$$

Plugging in the MLE for $\mathbf{W}$

When we plug in the MLE for $\mathbf{W}$ (with $\mathbf{R} = \mathbf{I}$) we find:

$$\mathbf{M} = \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})\mathbf{U}_L^\top + \sigma^2\mathbf{I}$$

and hence:

$$\mathbf{C}^{-1} = \sigma^{-2} \left[\mathbf{I} - \mathbf{U}_L(\mathbf{L}_L - \sigma^2\mathbf{I})\mathbf{\Lambda}_L^{-1}\mathbf{U}_L^\top \right]$$

$$\log |\mathbf{C}| = (D - L) \log \sigma^2 + \sum_{j=1}^L \log \lambda_j$$

Thus we can avoid all matrix inversions (since $\mathbf{\Lambda}_L^{-1} = \text{diag}(1/\lambda_j)$).

The encoder: posterior over latent variables

To use PPCA as an alternative to PCA, we need the posterior mean $\mathbb{E}[\mathbf{z}|\mathbf{x}]$ – the equivalent of the **encoder model**.

Using Bayes' rule for Gaussians:

$$p(\mathbf{z}|\mathbf{x}) = \mathcal{N}\left(\mathbf{z} \mid \mathbf{M}^{-1}\mathbf{W}^\top(\mathbf{x} - \boldsymbol{\mu}), \sigma^2\mathbf{M}^{-1}\right)$$

where $\mathbf{M}$ is as defined before.

In the $\sigma^2 = 0$ limit, the posterior mean using the MLE parameters becomes:

$$\mathbb{E}[\mathbf{z}|\mathbf{x}] = (\mathbf{W}^\top\mathbf{W})^{-1}\mathbf{W}^\top(\mathbf{x} - \bar{\mathbf{x}})$$

which is the **orthogonal projection** of the data into the latent space, as in standard PCA.

EM for (P)PCA

EM provides a useful alternative to eigenvector methods, and continues to work in the zero-noise limit $\sigma^2 = 0$.

Let:

- ▶ $\tilde{\mathbf{Z}} = \mathbf{Z}^\top$: $L \times N$ matrix of posterior means (low-dimensional representations)
- ▶ $\tilde{\mathbf{X}} = \mathbf{X}^\top$: original data, stored along its columns

When $\sigma^2 = 0$:

$$\tilde{\mathbf{Z}} = (\mathbf{W}^\top \mathbf{W})^{-1} \mathbf{W}^\top \tilde{\mathbf{X}}$$

This constitutes the **E step** – just an orthogonal projection of the data.

EM for (P)PCA: M step

The M step is given by:

$$\hat{\mathbf{W}} = \left[\sum_i \mathbf{x}_i \mathbb{E}[\mathbf{z}_i]^\top \right] \left[\sum_i \mathbb{E}[\mathbf{z}_i] \mathbb{E}[\mathbf{z}_i]^\top \right]^{-1}$$

since $\mathbf{\Sigma} = \text{Cov}[\mathbf{z}_i | \mathbf{x}_i, \boldsymbol{\theta}] = \mathbf{0}$ when $\sigma^2 = 0$.

Comparison: this is analogous to the MLE for multi-output linear regression

$$\mathbf{w} = \left(\sum_i \mathbf{y}_i \mathbf{x}_i^\top \right) \left(\sum_i \mathbf{x}_i \mathbf{x}_i^\top \right)^{-1}$$

The M step is like linear regression where the observed inputs are replaced by the expected values of the latent variables.

EM for (P)PCA: full algorithm

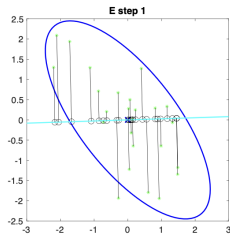
In summary, here is the entire algorithm:

$$\tilde{\mathbf{Z}} = (\mathbf{W}^\top \mathbf{W})^{-1} \mathbf{W}^\top \tilde{\mathbf{X}} \quad (\text{E step})$$

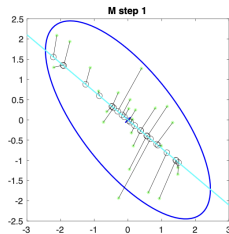
$$\mathbf{W} = \tilde{\mathbf{X}} \tilde{\mathbf{Z}}^\top (\tilde{\mathbf{Z}} \tilde{\mathbf{Z}}^\top)^{-1} \quad (\text{M step})$$

- ▶ The only stable fixed point of EM is the **globally optimal solution**.
- ▶ EM converges to $\mathbf{W}$ spanning the same linear subspace as the first L eigenvectors.
- ▶ For $\mathbf{W}$ orthogonal with eigenvectors in descending eigenvalue order, we must **orthogonalize** the result (cheap).
- ▶ Alternatively, EM can be modified to give the principal basis directly.

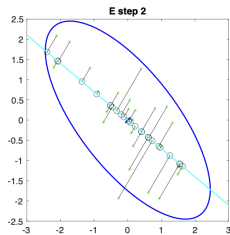
Illustration of EM for PCA when $D = 2$ and $L = 1$.



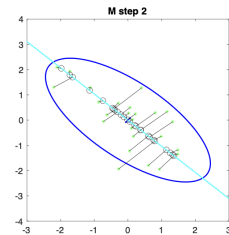
(a)



(b)



(c)



(d)

Advantages of EM for PCA

EM has the following advantages over eigenvector methods:

- ▶ **Faster**: assuming $N, D \gg L$, dominant cost is the projection in the E step, so overall time is $O(TLND)$, with T = number of iterations (often very small). This beats $O(\min(ND^2, DN^2))$ of naive eigenvector methods (though sophisticated methods like Lanczos are comparable).
- ▶ Can be implemented **online**: update **W** as data streams in.
- ▶ Can handle **missing data** simply.
- ▶ Can be extended to **mixtures of PPCA/FA** models.
- ▶ Can be modified to **variational EM** / **variational Bayes EM** for more complex models.

Dimension reduction

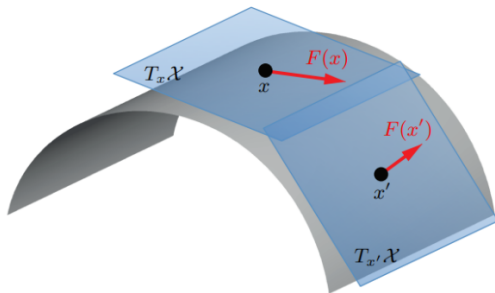
- ▶ Introduction
- ▶ PCA
- ▶ Factor analysis
 - ▶ Factor analysis and PCA: PPCA
- ▶ **Manifold learning**
 - ▶ Multi-dimensional scaling
 - ▶ tSNE
 - ▶ Manifold learning and PCA: kernel PCA

Manifold learning

- ▶ Goal: recover **low-dimensional structure** in **high-dimensional data**.
- ▶ The structure is often modeled as a **curved manifold**.
- ▶ This is known as **manifold learning** or **nonlinear dimensionality reduction**.
- ▶ Main uses: **unsupervised learning**, **visualization**, and **preprocessing** for supervised learning.

What are manifolds?

- ▶ A **manifold** is a topological space which is locally Euclidean.
- ▶ Example: the surface of the earth – a curved 2d surface embedded in 3d space. At each local point, the earth seems flat.
- ▶ Formally, a d -dimensional manifold $\mathcal{X}$ is a space in which each point $x \in \mathcal{X}$ has a neighborhood topologically equivalent to a d -dimensional Euclidean space, called the **tangent space**, denoted $\mathcal{T}_x = T_x\mathcal{X}$.



Riemannian manifold and the manifold hypothesis

Riemannian manifold

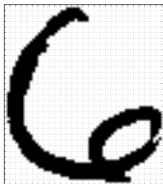
A differentiable manifold with an inner product operator at each point x in tangent space (depending smoothly on x).

- ▶ Induces a notion of distance, angles, volume.
- ▶ The collection of inner products is called a **Riemannian metric**.
- ▶ Any sufficiently smooth Riemannian manifold can be embedded into a (potentially higher-dimensional) Euclidean space.

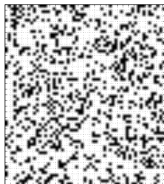
The manifold hypothesis

Most “naturally occurring” high-dimensional datasets lie on a low-dimensional manifold

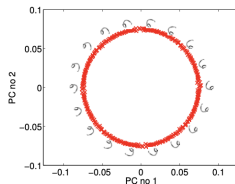
The manifold hypothesis – example



(a)



(b)



(c)

- a) An image of the digit 6, size 64×57 – 3648-dim. space (one dimension per pixel).
- b) Draw a random point from the space $\{0, 1\}^{3648}$ – does not look like a digit. However, pixels are not independent - they are generated by a lower-dimensional structure (the shape of the digit).
- c) A dataset created by rotating the original image by one degree 360 times. Applying PCA and projecting to 2d reveals a curved 2d manifold – **intrinsic dimensionality** d of the data is 2, even though the **ambient dimensionality** D is 3,648.

Dimension reduction

- ▶ Introduction
- ▶ PCA
- ▶ Factor analysis
 - ▶ Factor analysis and PCA: PPCA
- ▶ Manifold learning
 - ▶ **Multi-dimensional scaling**
 - ▶ tSNE
 - ▶ Manifold learning and PCA: kernel PCA

Multi-dimensional scaling (MDS)

- ▶ Goal: find low-dimensional vectors $\{\mathbf{z}_i \in \mathbb{R}^L : i = 1:N\}$ such that pairwise distances between these vectors are as similar as possible to a set of pairwise dissimilarities $\mathbf{D} = \{d_{ij}\}$ provided by the user.
- ▶ Several variants exist, one of which is equivalent to PCA.

Classical MDS

Given an $N \times D$ data matrix $\mathbf{X}$ with rows $\mathbf{x}_i$, define the centered Gram (similarity) matrix:

$$\tilde{K}_{ij} = \langle \mathbf{x}_i - \bar{\mathbf{x}}, \mathbf{x}_j - \bar{\mathbf{x}} \rangle$$

In matrix form, $\tilde{\mathbf{K}} = \tilde{\mathbf{X}}\tilde{\mathbf{X}}^\top$, where $\tilde{\mathbf{X}} = \mathbf{C}_N\mathbf{X}$ and $\mathbf{C}_N = \mathbf{I}_N - \frac{1}{N}\mathbf{1}_N\mathbf{1}_N^\top$ is the centering matrix.

Classical MDS: the strain objective

Define the **strain** of a set of embeddings:

$$\mathcal{L}_{\text{strain}}(\mathbf{Z}) = \sum_{i,j} \left(\tilde{K}_{ij} - \langle \tilde{\mathbf{z}}_i, \tilde{\mathbf{z}}_j \rangle \right)^2 = \|\tilde{\mathbf{K}} - \tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^\top\|_F^2$$

where $\tilde{\mathbf{z}}_i = \mathbf{z}_i - \bar{\mathbf{z}}$. Minimizing this loss is called **classical MDS**.

The best rank- L approximation to $\tilde{\mathbf{K}}$ is its truncated SVD $\tilde{\mathbf{K}} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$. Since $\tilde{\mathbf{K}}$ is positive semi definite, $\mathbf{V} = \mathbf{U}$, so:

$$\tilde{\mathbf{Z}}\tilde{\mathbf{Z}}^\top = \mathbf{U}\mathbf{S}\mathbf{U}^\top = (\mathbf{U}\mathbf{S}^{\frac{1}{2}})(\mathbf{S}^{\frac{1}{2}}\mathbf{U}^\top)$$

Thus the embedding vectors are the rows of $\tilde{\mathbf{Z}} = \mathbf{U}\mathbf{S}^{\frac{1}{2}}$.

Classical MDS from distances only

If we only have Euclidean distances (not raw features), compute the squared distance matrix $\mathbf{D}^{(2)} = \mathbf{D} \odot \mathbf{D}$:

$$D_{ij}^{(2)} = \|\mathbf{x}_i - \mathbf{x}_j\|^2 = \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2 + \|\mathbf{x}_j - \bar{\mathbf{x}}\|^2 - 2\tilde{K}_{ij}$$

$\mathbf{D}^{(2)}$ differs from $\tilde{\mathbf{K}}$ only by row/column constants (and a factor of -2). By double centering:

$$\tilde{\mathbf{K}} = -\frac{1}{2}\mathbf{C}_N\mathbf{D}^{(2)}\mathbf{C}_N$$

i.e.,

$$\tilde{K}_{ij} = -\frac{1}{2} \left(d_{ij}^2 - \frac{1}{N} \sum_{l=1}^N d_{il}^2 - \frac{1}{N} \sum_{l=1}^N d_{jl}^2 + \frac{1}{N^2} \sum_{l=1}^N \sum_{m=1}^N d_{lm}^2 \right)$$

We then compute embeddings as before.

Metric and non-metric MDS

Metric MDS: minimize

$$\mathcal{L}(\mathbf{Z}) = \sum_{i < j} (d_{ij} - \hat{d}_{ij})^2, \quad \hat{d}_{ij} = \|\mathbf{z}_i - \mathbf{z}_j\|$$

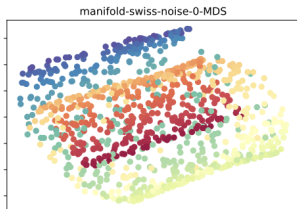
- ▶ A different objective than classical MDS – results differ even if d_{ij} are Euclidean.
- ▶ Solved via a bound optimization algorithm.

Non-metric MDS: match the *ranking* of similarities rather than distances. Let $f(d)$ be a monotonic transform from distances to ranks:

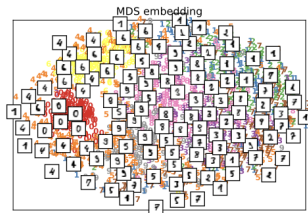
$$\mathcal{L}_{\text{NM}}(\mathbf{Z}) = \sqrt{\frac{\sum_{i < j} (f(d_{ij}) - \hat{d}_{ij})^2}{\sum_{ij} \hat{d}_{ij}^2}}$$

Optimized iteratively: (1) fit f via isotonic regression for fixed $\mathbf{Z}$; (2) optimize $\mathbf{Z}$ via gradient descent for fixed f ; repeat.

Metric MDS applied to running examples



(a)



(b)

Dimension reduction

- ▶ Introduction
- ▶ PCA
- ▶ Factor analysis
 - ▶ Factor analysis and PCA: PPCA
- ▶ Manifold learning
 - ▶ Multi-dimensional scaling
 - ▶ **tSNE**
 - ▶ Manifold learning and PCA: kernel PCA

t-SNE: introduction

t-SNE is a popular nonconvex technique for learning low-dimensional embeddings, extending **stochastic neighbor embedding (SNE)**

Stochastic Neighbor Embedding (SNE)

Convert high-dimensional Euclidean distances into conditional probabilities representing similarities:

$$p_{j|i} = \frac{\exp\left(-\frac{1}{2\sigma_i^2}\|\mathbf{x}_i - \mathbf{x}_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\frac{1}{2\sigma_i^2}\|\mathbf{x}_i - \mathbf{x}_k\|^2\right)}$$

- ▶ $p_{j|i}$ – the probability that point i would pick point j as its neighbor if neighbors we picked in proportion under a Gaussian centered at $\mathbf{x}_i$
- ▶ σ_i^2 is the variance for point i , used to “magnify” scale in dense regions and diminish it in sparse regions.

SNE: objective and gradient

Define similarities in the low-dimensional embedding analogously:

$$q_{j|i} = \frac{\exp(-\|\mathbf{z}_i - \mathbf{z}_j\|^2)}{\sum_{k \neq i} \exp(-\|\mathbf{z}_i - \mathbf{z}_k\|^2)}$$

The variance is fixed to a constant – changing it would just rescale the map, not its topology.

Good embedding: $q_{j|i}$ and $p_{j|i}$ are close $\rightarrow$ **SNE objective**: match $q_{j|i}$ to $p_{j|i}$ via KL divergence:

$$\mathcal{L} = \sum_i D_{\text{KL}}(P_i \| Q_i) = \sum_i \sum_j p_{j|i} \log \frac{p_{j|i}}{q_{j|i}}$$

This is **asymmetric**: large cost if small $q_{j|i}$ models a large $p_{j|i}$ – prefers pulling distant points together over pushing nearby points apart, cf. its gradient

$$\nabla_{\mathbf{z}_i} \mathcal{L}(\mathbf{Z}) = 2 \sum_j (\mathbf{z}_i - \mathbf{z}_j) (p_{j|i} - q_{j|i} + p_{i|j} - q_{i|j})$$

Symmetric SNE

A simpler version minimizes a single KL between the joint distributions P (high-d) and Q (low-d):

$$\mathcal{L} = D_{\text{KL}}(P\|Q) = \sum_{i < j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad \text{where}$$

$$p_{ij} = \frac{\exp\left(-\frac{1}{2\sigma^2}\|\mathbf{x}_i - \mathbf{x}_j\|^2\right)}{\sum_{k \neq l} \exp\left(-\frac{1}{2\sigma^2}\|\mathbf{x}_k - \mathbf{x}_l\|^2\right)}$$

and q_{ij} defined similarly. The gradient becomes:

$$\nabla_{\mathbf{z}_i} \mathcal{L}(\mathbf{Z}) = 4 \sum_j (\mathbf{z}_i - \mathbf{z}_j)(p_{ij} - q_{ij})$$

- ▶ Easier to implement, but loses the property that the data is its own optimal embedding when $L = D$.
- ▶ Performs similarly in practice when $L \ll D$.

t-distributed SNE (t-SNE)

Crowding problem: SNE and similar techniques tend to squeeze relatively distant high-dim. points close together in low-dim. embedding space, due to squared errors / Gaussian probabilities.

Solution: use a heavier-tailed distribution in latent space – the Student- t with $\nu = 1$ (i.e., Cauchy):

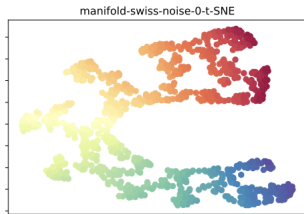
$$q_{ij} = \frac{(1 + \|\mathbf{z}_i - \mathbf{z}_j\|^2)^{-1}}{\sum_{k < l} (1 + \|\mathbf{z}_k - \mathbf{z}_l\|^2)^{-1}}$$

Using the same global KL objective as for SNE, the gradient is:

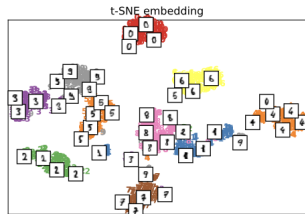
$$\nabla_{\mathbf{z}_i} \mathcal{L} = 4 \sum_j (p_{ij} - q_{ij})(\mathbf{z}_i - \mathbf{z}_j)(1 + \|\mathbf{z}_i - \mathbf{z}_j\|^2)^{-1}$$

- ▶ Symmetric (Gaussian) SNE has the same gradient but lacks the $(1 + \|\mathbf{z}_i - \mathbf{z}_j\|^2)^{-1}$ term.
- ▶ This term acts like an **inverse square law**: embedded points behave like stars within well-separated galaxies (clusters) – useful for unsupervised class separation.

tSNE applied to running examples



(a)



(b)

Dimension reduction

- ▶ Introduction
- ▶ PCA
- ▶ Factor analysis
 - ▶ Factor analysis and PCA: PPCA
- ▶ Manifold learning
 - ▶ Multi-dimensional scaling
 - ▶ tSNE
 - ▶ **Manifold learning and PCA: kernel PCA**

Kernel PCA: idea

PCA (and classical MDS) find the best **linear** projection preserving pairwise similarities. Kernel PCA extends this to **nonlinear** projections.

- ▶ PCA can be solved via eigenvectors of the Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^\top$.
- ▶ **Kernel trick**: replace inner products $\mathbf{x}_i^\top \mathbf{x}_j$ with a kernel $K_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$.
- ▶ By Mercer's theorem, this implicitly maps $\mathbf{x}_i \mapsto \phi(\mathbf{x}_i)$ into a (possibly infinite-dimensional) feature space, where ordinary PCA is performed.
- ▶ We never compute ϕ_i explicitly – everything is expressed via $\mathbf{K}$, and new points are projected using $\mathbf{k}_* = [\mathcal{K}(\mathbf{x}_*, \mathbf{x}_1), \dots, \mathcal{K}(\mathbf{x}_*, \mathbf{x}_N)]$.
- ▶ A technical detail (**double centering**) is needed since ϕ_i can't be mean-subtracted directly.

Kernel PCA: remarks

- ▶ A **linear kernel** recovers regular PCA / classical MDS ($L \leq D$).
- ▶ A **non-degenerate kernel** allows up to N components, since the feature space dimension D^* can exceed D (even be infinite).
- ▶ With an **RBF kernel**, leading components tend to separate clusters, later components split them further.
- ▶ In practice, kPCA with RBF often **expands** rather than reduces the feature space – so it's not always useful as a dimensionality-reduction tool, despite the elegant theory.

References

- ▶ [ESLII](#): Chapter 14.5, 14.6
- ▶ [PML](#): Chapter 20